

SORTING ALGORITHM

Priority	Algorithm	Best	Average	Worst	Space	Stable
★★★★★	Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
★★★★★	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
★★★★★	Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
★★★★★	Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
★★★★★	Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	✗
★★★★★	Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗

1. Bubble Sort

Definition

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

The largest element gradually "**bubbles**" to the end of the array.

Example:

5 3 4 1 5 3 → swap

3 5 4 1 5 4 → swap

3 4 5 1 5 1 → swap

3 4 1 5

```
public static void bubbleSort(int[] arr) {  
  
    for (int i = 0; i < arr.length - 1; i++) {  
  
        for (int j = 0; j < arr.length - 1 - i; j++) {  
  
            if (arr[j] > arr[j + 1]) {  
  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

```

</> Java

public static void bubbleSort(int[] arr) {

    int n = arr.length;

    for (int i = 0; i < n - 1; i++) {

        boolean swapped = false;

        for (int j = 0; j < n - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                swapped = true;
            }
        }

        if (!swapped)
            break;
    }
}

```

Complexity

Best : $O(n)$

Average : $O(n^2)$

Worst : $O(n^2)$

Space : $O(1)$

Placement point

If the interviewer asks:

"How can you optimize Bubble Sort?"

Answer:

We can use a swapped flag. If no swapping happens during a complete pass, the array is already sorted, so we can stop early.

2. Selection Sort

Definition

Selection Sort repeatedly finds the **minimum element** from the unsorted part and places it at the beginning.

Example:

5 3 4 1

Minimum = 1

1 3 4 5

Then find the minimum from the remaining portion.

```
</> Java

public static void selectionSort(int[] arr) {

    int n = arr.length;

    for (int i = 0; i < n - 1; i++) {

        int minIndex = i;

        for (int j = i + 1; j < n; j++) {

            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

Complexity

Best : $O(n^2)$

Average : $O(n^2)$

Worst : $O(n^2)$

Space : $O(1)$

3. Insertion Sort

Definition Insertion Sort builds the sorted array **one element at a time**. Think about arranging playing cards in your hand.

5 3 4 1

Take 3:

3 5 4 1

Take 4:

3 4 5 1

Take 1:

1 3 4 5

```
</> Java

public static void insertionSort(int[] arr) {

    for (int i = 1; i < arr.length; i++) {

        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {

            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}
```

Complexity

Best : $O(n)$

Average : $O(n^2)$

Worst : $O(n^2)$

Space : $O(1)$

4. Merge Sort ★

Definition

Merge Sort uses **Divide and Conquer**.

It:

1. Divides the array into two halves.
2. Recursively sorts both halves.
3. Merges the sorted halves.

8 3 5 2

```
      [8 3 5 2]
      /   \
    [8 3] [5 2]
    / \   / \
   8  3 5  2
```

After sorting:

```
    [3 8]   [2 5]
      ↓
    [2 3 5 8]
```

// Java

```
public static void mergeSort(int[] arr, int left, int right) {

    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}
```

```

public static void merge(int[] arr, int left, int mid, int right) {

    int[] temp = new int[right - left + 1];

    int i = left;
    int j = mid + 1;
    int k = 0;

    while (i <= mid && j <= right) {

        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }

    for (int x = 0; x < temp.length; x++) {
        arr[left + x] = temp[x];
    }
}

```



Call: mergeSort(arr, 0, arr.length - 1);

Complexity

Best : $O(n \log n)$

Average : $O(n \log n)$

Worst : $O(n \log n)$

Space : $O(n)$

Interview question

Why is Merge Sort $O(n \log n)$?

Because:

Number of levels = $\log n$

Work at each level = n

Therefore:

$O(n \times \log n)$

$= O(n \log n)$

5. Quick Sort ★★

Definition

Quick Sort is also a **Divide and Conquer** algorithm.

It chooses a **pivot**, places the pivot in its correct position, and recursively sorts the left and right parts.

```
static void quickSort(int[] arr, int low, int high) {  
  
    if (low < high) {  
  
        int pivot = partition(arr, low, high);  
  
        quickSort(arr, low, pivot - 1);  
  
        quickSort(arr, pivot + 1, high);  
    }  
}
```

```
static int partition(int[] arr, int low, int high) {  
  
    int pivot = arr[high];  
  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
  
        if (arr[j] < pivot) {  
  
            i++;  
  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
  
    int temp = arr[i + 1];  
    arr[i + 1] = arr[high];  
    arr[high] = temp;  
  
    return i + 1;  
}
```

Call:

```
quickSort(arr, 0, arr.length - 1);
```

Complexity

Best : $O(n \log n)$

Average : $O(n \log n)$

Worst : $O(n^2)$

Space:

Average : $O(\log n)$

Worst : $O(n)$